

操作系统

第二章 进程管理

1、进程的表示



1、进程图像（不全）

- 进程控制块（Process Control Block, PCB）+ 代码 + 数据
- PCB，最重要的内核数据结构，登记应用程序的执行状态
- 代码+数据，包括应用程序的所有逻辑段



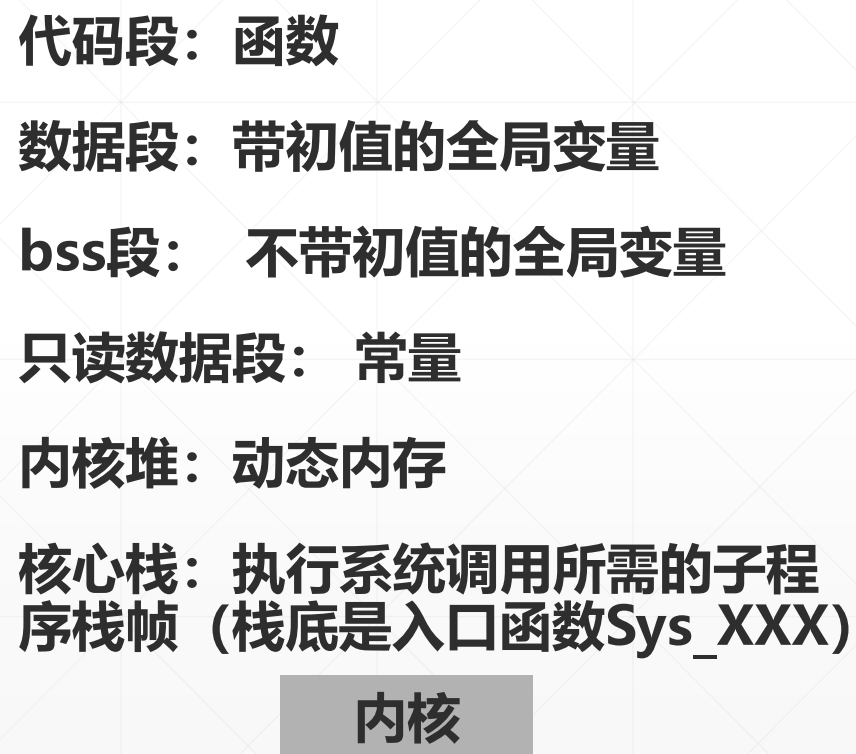
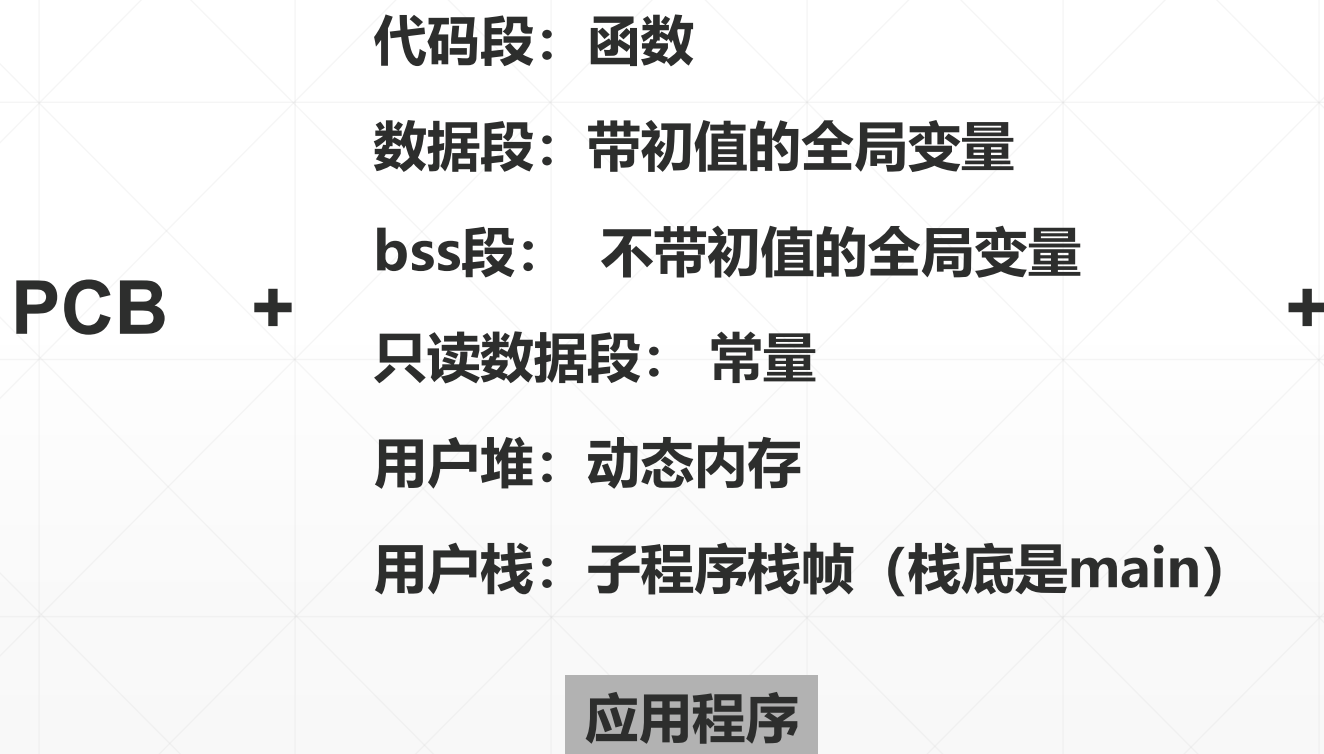
每个进程有两种执行状态 (Unix系统)

- 用户态，执行应用程序，访问应用程序的所有逻辑段。
- 核心态，执行操作系统内核，访问内核代码 和 数据。

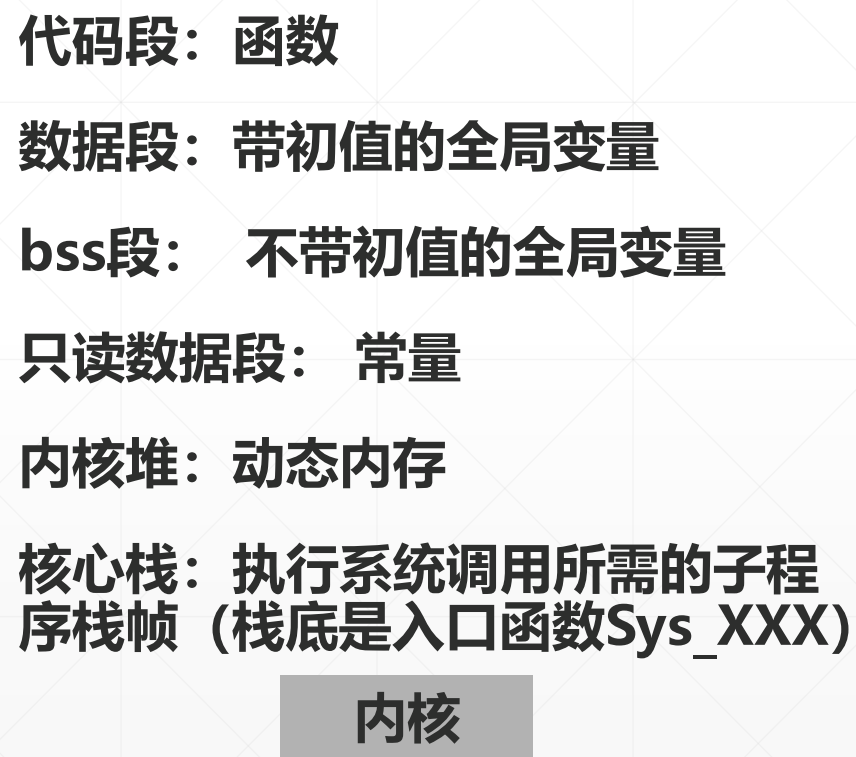
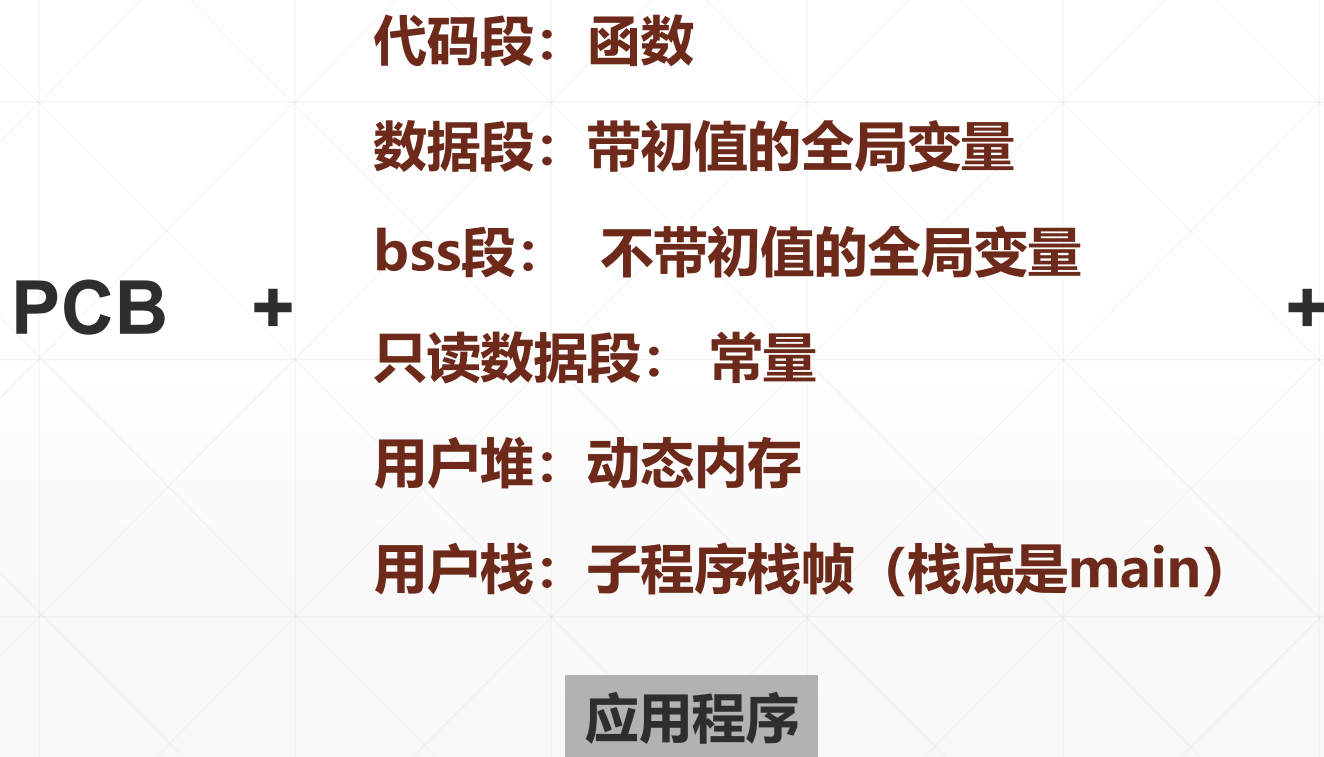
用户栈 和 核心栈 (Unix系统)

- 执行应用程序时，使用用户栈。
- 执行系统调用时，使用核心栈。

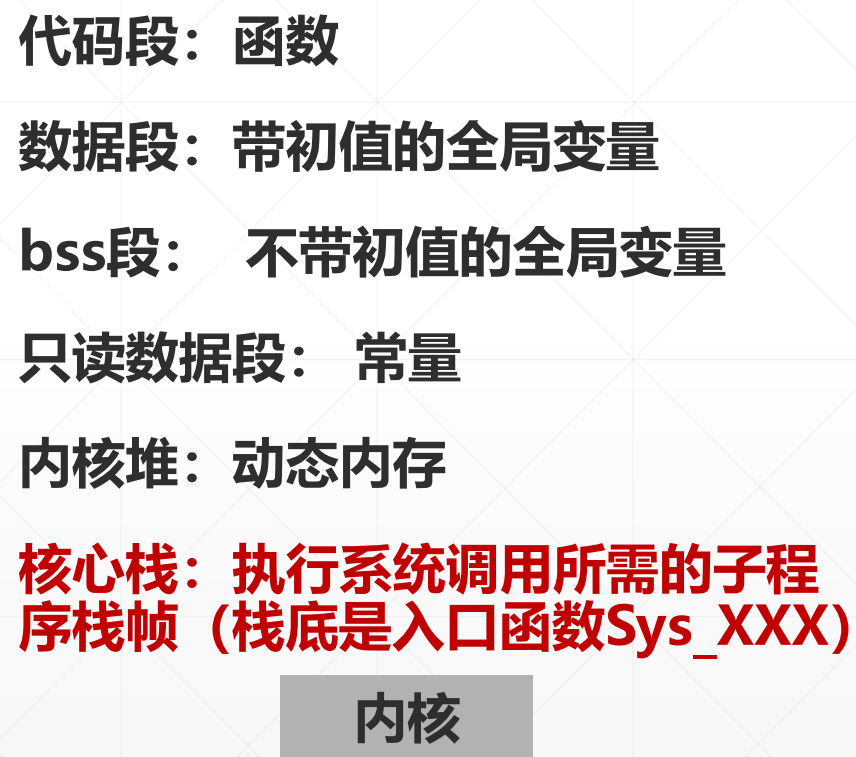
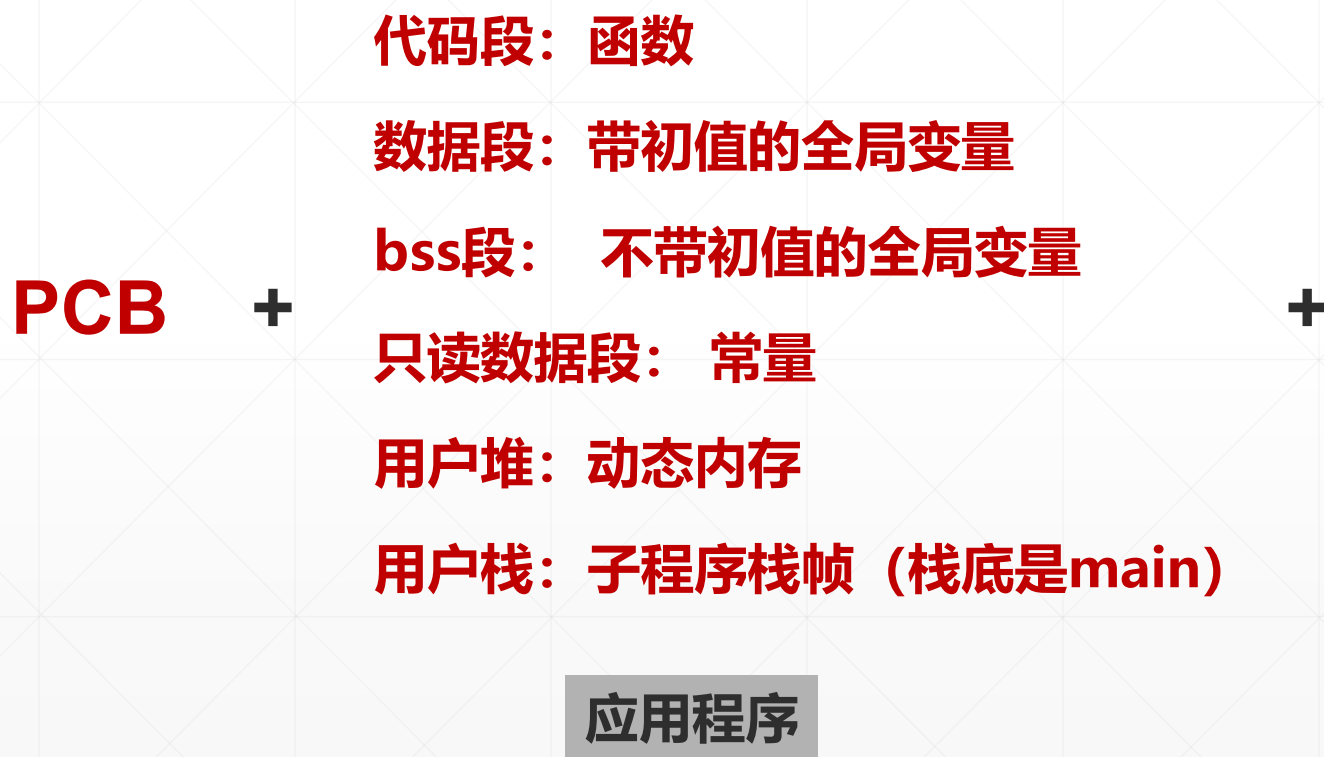
进程图像



进程图像 (用户空间 & 核心空间)



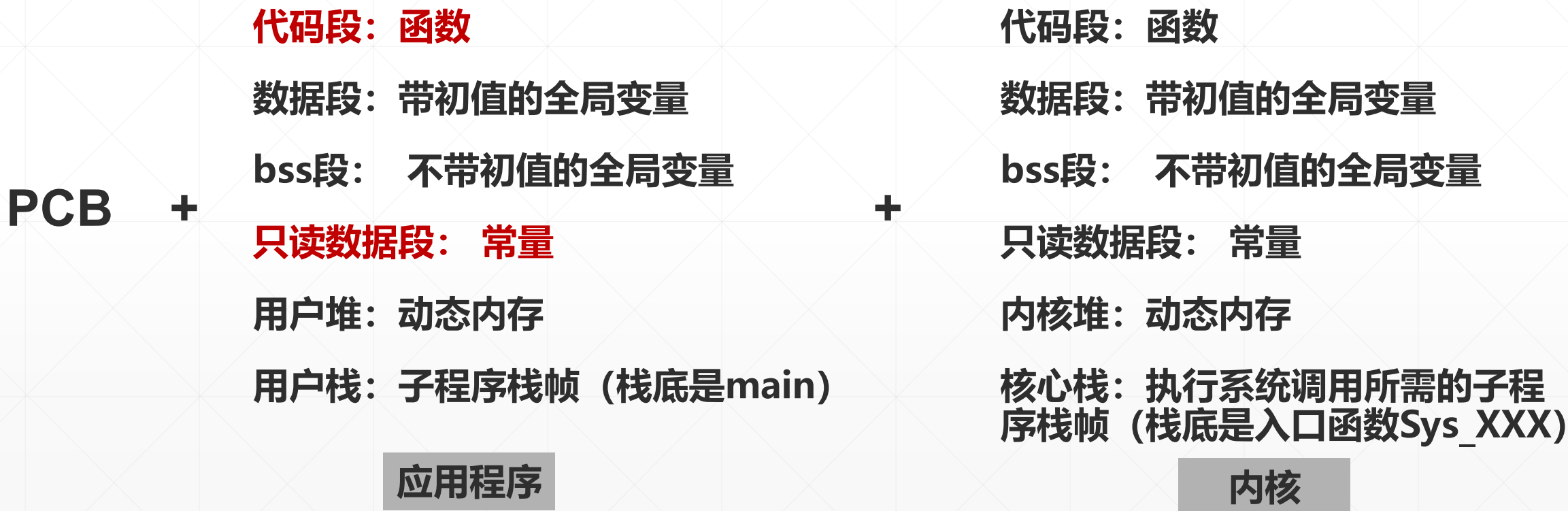
进程图像 (进程私有 & 所有进程共用)



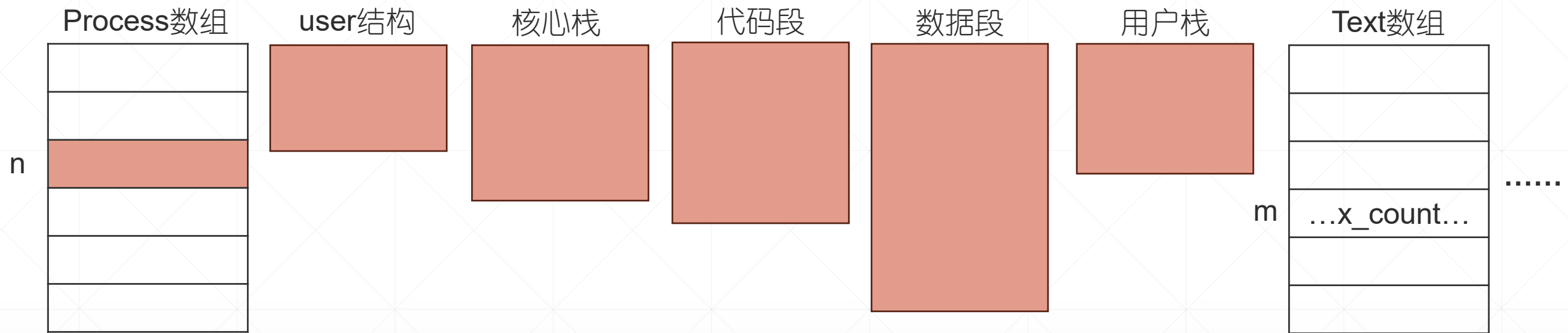


进程图像（执行同一个应用程序的所有进程共享正文段）

因为程序执行时，不会更改代码 和 只读数据



Unix V6++ 进程图像 (私有)



进程的**PCB**: `Process[n] + user`

执行同一个应用程序的所有进程 共享**代码段**。
代码段由代码段控制块**Text**结构管理，字段 `x_count` 是共享这个代码段的进程数量。

进程**数据段**，合并应用程序的数据段，只读数据段，`bss`段 和 用户堆。
进程**用户栈**，应用程序的用户栈。

Unix V6+ + 进程图像 (所有进程共享)

内核静态部分

内核函数
(系统调用入口程序.....)

全局变量
(Process数组, Text数组)

.....

.....

内核堆 (未用)

动态内存

内核动态部分
(用前分配, 用完释放)



2、Unix V6++，怎么摆放每块进程图像

1、虚空间中

2、物理空间中

进程的虚地址空间

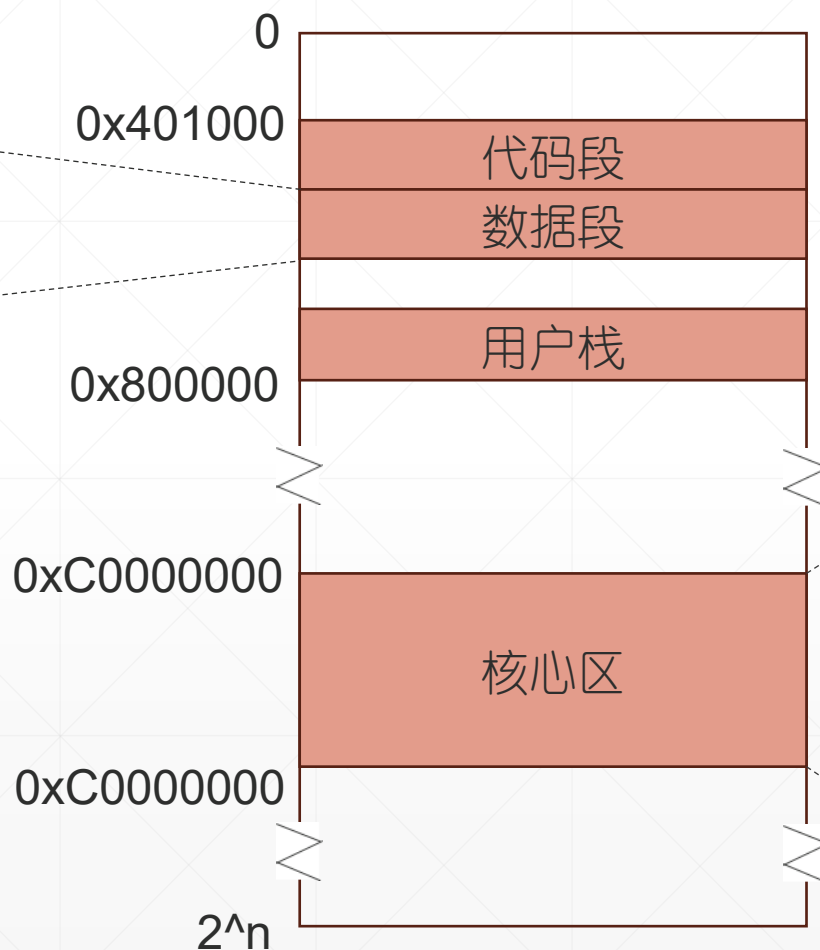
- 进程，能够访问的所有指令（代码）、数据的地址，组成的集合。
- 虚地址空间分2块，泾渭分明。
 - 低地址区放应用程序（用户区）
 - 高地址区放操作系统内核（核心区）



- 具体到一台具体计算机，所有进程虚地址空间尺寸相等，格式统一。
- 每个进程享有 2^n 字节的虚空间， n 是CPU的位数。
 - 32位机， $n=32$ ，虚空间 2^{32} (4G字节);
 - 64位机， $n=64$ ，虚空间 2^{64} (天文数字);
 Unix V6++运行其上的i386是32位机
- Unix V6++进程
 - 用户区， $[0, 0x800000)$
// 十进制表示 $[0, 8M)$
 - 核心区， $[0xC0000000, 0xC0400000)$
// 十进制表示 $[3G, 3G+4M)$
- 32位Linux进程
 - 用户区， $[0, 0xC0000000)$
// 十进制表示 $[0, 3G)$
 - 核心区， $[0xC0000000, 0x100000000)$
// 十进制表示 $[3G, 4G)$

Unix V6+ + 进程的虚地址空间布局

进程数据段，依次排列应
用程序 数据段、只读数
据段、bss段、堆（若有）



[0x401000, 0x401000 + 代码段长度)

[0x401000 + 代码段长度,
0x401000 + 代码段长度 + 数据段长度)

[0x800000 - 用户栈长度, 0x800000)

3G

null

代码, 全局变量

[3G+1M , 3G+1.5M)

内核堆

[3G+1.5M , 3G+2M)

页表区

[3G+2M , 3G+4M-4K)

User + 核心栈

[3G+4M-4K, 3G+4M)

作业：用16进制数表示区间

ShowStack进程的代码段 和 数据段，它的静态部分

```
#include <stdio.h>
```

```
int version=1;
```

```
main1()
```

```
{
    int a,b,result;
    a = 1;
    b = 2;
    result = sum(a,b);
    printf("result=%d\n",result);
}
```

```
int sum(var1, var2)
```

```
{
    int count;
    version = 2;
    count = var1 + var2;
    return(count);
}
```

```
D:\UNIX V6++V1\oos\tools>objdump -h showStack.exe

showStack.exe:      file format pei-i386

Sections:
Idx Name              Size  ★  VMA  ★  LMA      File off  Algn
 0  .text              000020e8 00401000 00401000 00000400 2**2
    CONTENTS, ALLOC, LOAD, READONLY, CODE
 1  .data              0000000c 00404000 00404000 00002600 2**2
    CONTENTS, ALLOC, LOAD, DATA
 2  .rdata             00000204 00405000 00405000 00002800 2**3
    CONTENTS, ALLOC, LOAD, READONLY, DATA
 3  .bss               00000020 00406000 00406000 00000000 2**2
    ALLOC
```

代码段，3页
(每页4096字节)

数据段，3页

应用程序的逻辑段
VMA，首地址；Size 4096上取整，长度

ShowStack进程的用户栈， 它的动态部分

```
#include <stdio.h>
```

```
int version=1;
```

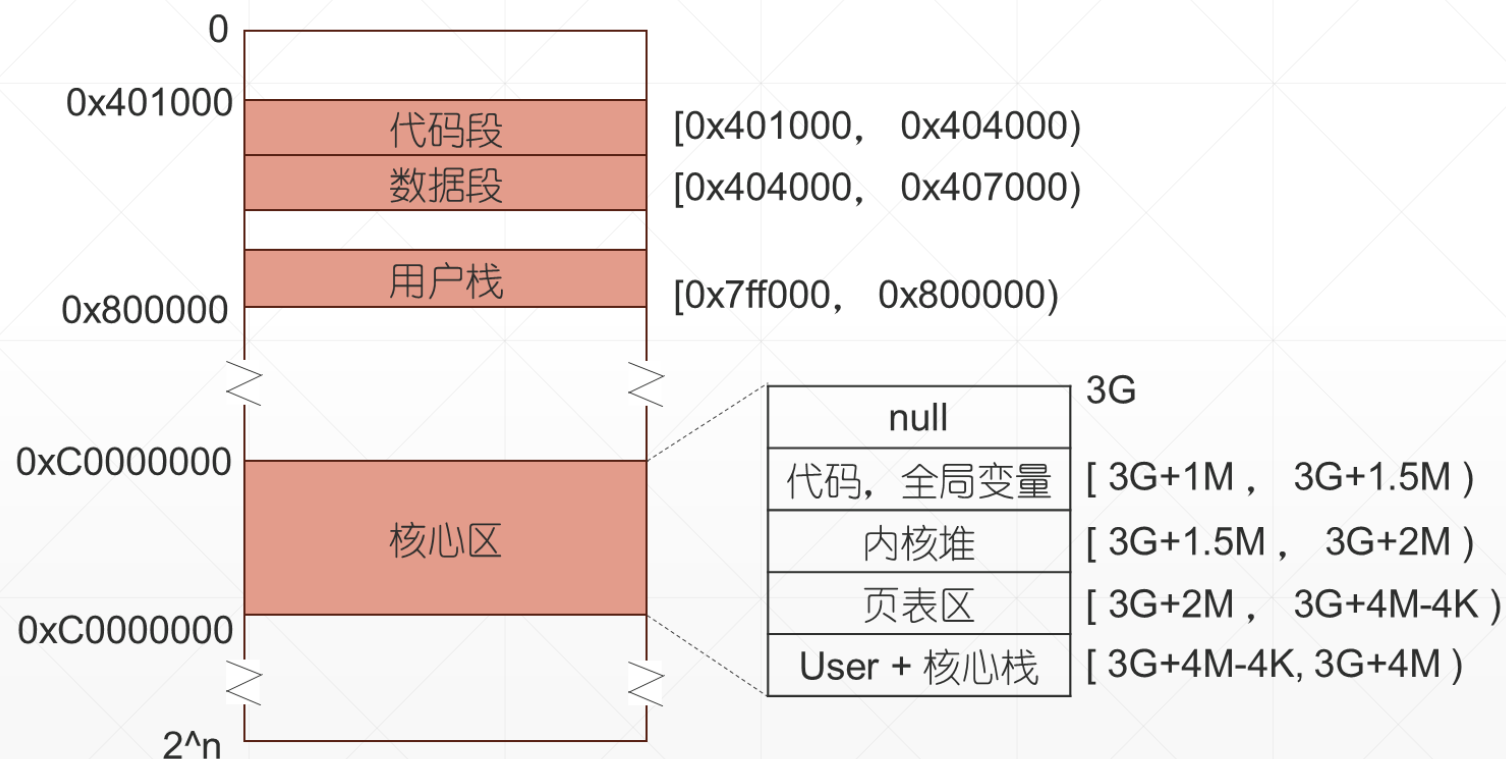
```
main1()
```

```
{
    int a,b,result;
    a = 1;
    b = 2;
    result = sum(a,b);
    printf("result=%d\n",result);
}
```

```
int sum(var1, var2)
```

```
{
    int count;
    version = 2;
    count = var1 + var2;
    return(count);
}
```

用户栈底, 0x800000; 初始堆栈一页



Unix V6++ 进程图像的动态变化

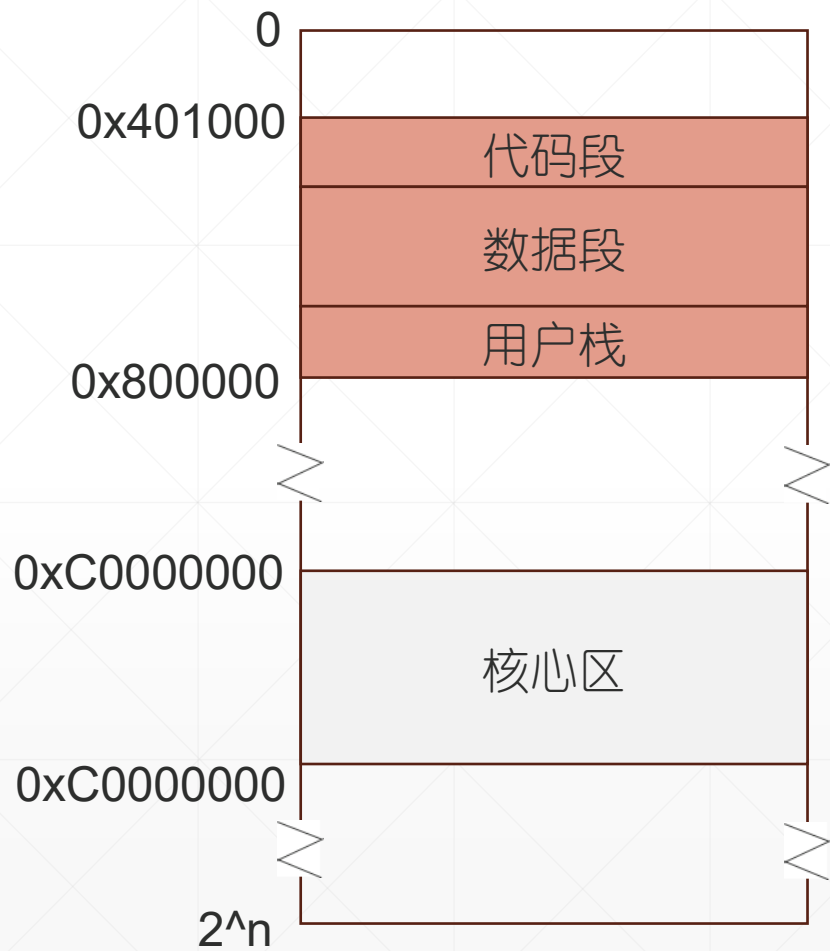
堆空间的变化

- 执行malloc申请动态内存块，若没有空间（数据段中的堆区域找不到足够大的空闲区域），内核会扩展进程的数据段，向高地址方向伸展，一次扩展3页→→→进程数据段，尺寸+=3*4096字节。
- 执行free，进程释放动态内存块。若导致进程数据段最高地址端出现尺寸为6页的大块空闲区域，缩短数据段，尺寸-=6*4096字节。

栈空间的变化

- 子程序调用，若 $ESP < 0x800000 - 0x1000$ (8M-4K)，初始堆栈溢出，扩展用户栈，向低地址方向生长，每次1页→→→进程用户栈，尺寸+=4096字节，首地址-=4096字节。

OOM (Out Of Memory)

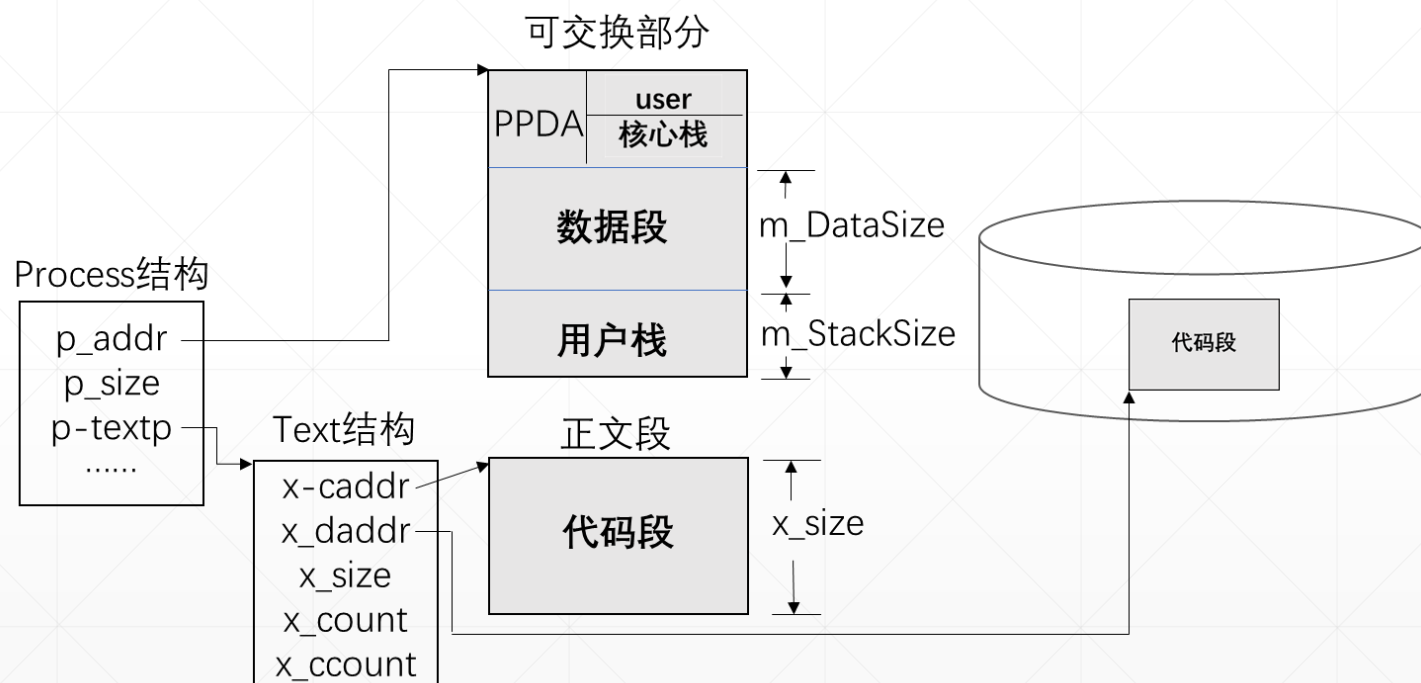


OOM, 虚空间 (用户空间) 用完了。

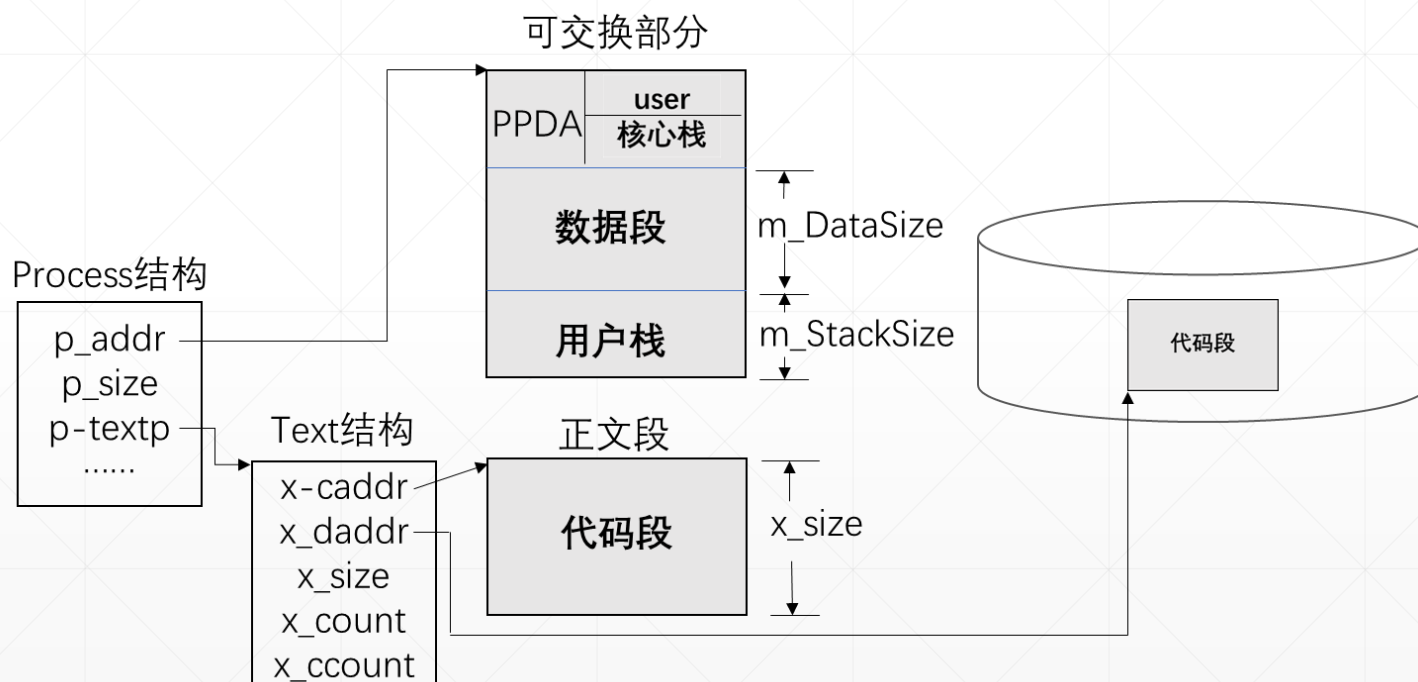
这个错误会导致进程夭折。

判断条件: 代码段长度 + 数据段长度 + 用户栈长度
 $> 0x800000 - 0x401000$

为进程图像分配的物理内存



- **Process结构**存放在Process数组。
- **共享正文段**，包括代码段和只读数据段（Unix V6++ buggy，把只读数据段放在了可交换区），可与执行同一应用程序的其它进程共享。这是为了节省进程图像的内存消耗。
- **可交换部分**，包括PPDA区，数据段和用户栈。PPDA（Per Process Data Area）区，4096字节，低地址端是user结构，高地址端是进程的核心栈。这块内存禁止其它进程访问。



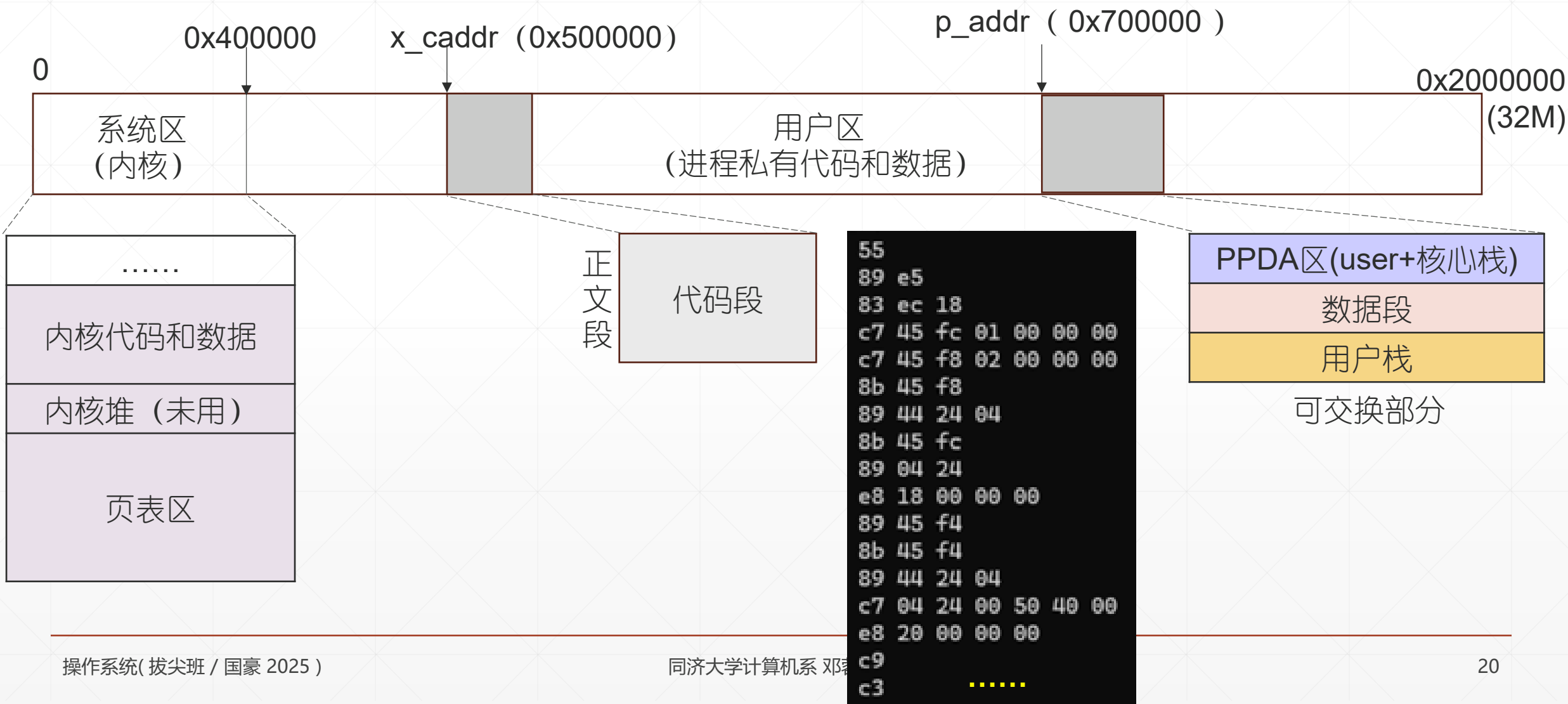
- Process结构（进程基本控制块）常驻内存。
- 代码段，内存、磁盘各存一个副本。
- 内存紧张时，进程的可交换部分可暂存于磁盘，以释放内存空间。必要时，还会释放正文段占据的内存。磁盘上，用来存放代码段和可交换部分的区域，叫做盘交换区。
- 代码段、可交换部分全部在内存，进程有执行的权力（进程图像在内存中）。否则，进程图像在盘交换区上，进程不可执行。

物理内存中的进程图像

- Unix V6++ 的物理内存



物理内存中的进程图像



地址映射



CPU执行单元 (EIP, `0x401000`)

`0x401000`

地址映射

`0x500000`

内存

正文段

代码段

55

```

55 ★
89 e5
83 ec 18
c7 45 fc 01 00 00 00
c7 45 f8 02 00 00 00
8b 45 f8
89 44 24 04
8b 45 fc
89 04 24
e8 18 00 00 00
89 45 f4
8b 45 f4
89 44 24 04
c7 04 24 00 50 40 00
e8 20 00 00 00
c9
c3
  
```

地址映射流程和实现

